

SonarQube Integration Project Submission

(Address Book Module)

1) Project Shared

I have integrated SonarQube Community Build with my Java project:

- ✓ **ProjectName:** AddressBookSystem
- ✓ **Module Included:** Address Book Module
- ✓ **Type:** Core Java + Eclipse Project
- ✓ **Source Folder:** src
- ✓ **Compiled Output Folder:** bin

2) Screenshots Provided

I have attached screenshots of:

A) Project Execution

Output of AddressBookSystem running successfully in Eclipse / Console

```
=====Welcome to Address Book Management System=====
====
What Would You want to do:
0. Exit Book Address
1. Create Address Book
2. Select Address Book
3. Add New contact
4. Edit Contact
5. Display Contacts
6. Delete a Person
7. Search Person by City
Enter a Task Number:
```

B) SonarQube Report

- SonarQube Dashboard showing:
 - Quality Gate Status: **Passed**
 - Maintainability Issues
 - Coverage and Duplication Summary

SonarQube community Projects Issues Rules Quality Profiles Quality Gates Administration More ▾

Address Book / main ✓ ▾ ?

Overview Issues Security Hotspots Code Measures Activity Project Settings ▾ Project Information

Quality Gate ○ Last analysis 7 minutes ago

✓ **Passed**

⚠ The last analysis has warnings. [See details](#)

New Code Overall Code

Security 0 Open issues A	Reliability 0 Open issues A	Maintainability 62 Open issues A
Accepted issues 0 🔍 Valid issues that were not fixed	Coverage 0.0% 🔴 On 260 lines to cover.	Duplications 8.3% 🟡 On 564 lines.
Security Hotspots 0 A		

SonarQube community Projects Issues Rules Quality Profiles Quality To exit full screen, press and hold Esc

Address Book / main ✓ ▾ ?

Overview Issues Security Hotspots Code Measures Activity Project Settings ▾ Project Information

Valid issues that were not fixed On 260 lines to cover. On 564 lines.

Security Hotspots
0 A

Activity

Graph type Issues ▾

There isn't enough data to generate an activity graph.

February 9, 2026 at 11:06 AM not provided Quality Gate: ✓ Passed

First analysis: 62 Issues • 0.0% Coverage • 8.3% Duplications

[See full history of analyses](#)

C) Sonar Scanner Execution

- Command Prompt output showing:
 - **ANALYSIS SUCCESSFUL**
 - Dashboard URL
[\(http://localhost:9000/dashboard?id=address-book&codeScope=overall\)](http://localhost:9000/dashboard?id=address-book&codeScope=overall)

```

11:06:16.081 INFO Sensor TextAndSecretsSensor [text]
11:06:16.121 INFO Available processors: 12
11:06:16.122 INFO Using 12 threads for analysis.
11:06:16.780 INFO The property "sonar.tests" is not set. To improve the analysis accuracy, we categorize a file as a test file if any of the following is true:
* The filename starts with "test"
* The filename contains "test." or "tests."
* Any directory in the file path is named: "doc", "docs", "test", "tests", "mock" or "mocks"
* Any directory in the file path has a name ending in "test" or "tests"

11:06:16.830 INFO Start fetching files for the text and secrets analysis
11:06:17.130 INFO Using Git CLI to retrieve dirty files
11:06:17.224 WARN Retrieving only language associated files, make sure to run the analysis inside a git repository to make use of inclusions specified via "sonar.text.inclusions"
11:06:17.226 INFO Starting the text and secrets analysis
11:06:17.231 INFO 7 source files to be analyzed for the text and secrets analysis
11:06:17.277 INFO 7/7 source files have been analyzed for the text and secrets analysis
11:06:17.281 INFO Sensor TextAndSecretsSensor [text] (done) | time=1200ms
11:06:17.293 INFO ----- Run sensors on project
11:06:17.591 INFO Sensor JavaProjectSensor [java]
11:06:17.596 INFO Sensor JavaProjectSensor [java] (done) | time=4ms
11:06:17.598 INFO Sensor Zero Coverage Sensor
11:06:17.627 INFO Sensor Zero Coverage Sensor (done) | time=30ms
11:06:17.629 INFO Sensor Java CPD Block Indexer
11:06:17.728 INFO Sensor Java CPD Block Indexer (done) | time=100ms
11:06:17.728 INFO ----- Gather SCA dependencies on project
11:06:17.736 INFO Dependency analysis skipped
11:06:17.740 INFO SCM Publisher No SCM system was detected. You can use the 'sonar.scm.provider' property to explicitly specify it.
11:06:17.747 INFO CPD Executor 1 file had no CPD blocks
11:06:17.749 INFO CPD Executor Calculating CPD for 6 files
11:06:17.776 INFO CPD Executor CPD calculation finished (done) | time=29ms
11:06:17.993 INFO Analysis report generated in 193ms, dir size=293.7 kB
11:06:18.369 INFO Analysis report compressed in 375ms, zip size=46.3 kB
11:06:18.738 INFO Analysis report uploaded in 368ms
11:06:18.742 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=address-book
11:06:18.744 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
11:06:18.745 INFO More about the report processing at http://localhost:9000/api/ce/task?id=8524bd10-52ae-4bde-9b0b-2a1c24202cea
11:06:18.783 INFO Analysis total time: 13.946 s
11:06:18.785 INFO SonarScanner Engine completed successfully
11:06:18.967 INFO EXECUTION SUCCESS
11:06:18.969 INFO Total time: 32.999s

```

3) Step-by-Step Procedure Followed (SonarQube Integration)

Step 1: Install SonarQube

1. Download SonarQube Community Build for Windows
2. Extract it to a folder (example): C:\sonarqube-26.2.0\
3. Open Command Prompt as Administrator
4. Go to: sonarqube\bin\windows-x86-64\
5. Run: StartSonar.bat

Step 2: Verify SonarQube is Running

1. Open browser: <http://localhost:9000>
2. Login using default credentials:
 - Username: **admin**
 - Password: **admin**
3. Set new password after login

Step 3: Create a New Project in SonarQube

1. Click **Create Project**
2. Enter Project Key: **address-book**
3. Generate a token (used for scanner authentication)

Step 4: Download SonarScanner

1. Download SonarScanner CLI for Windows
2. Extract it (example): **C:\sonar-scanner-8.0.1**
3. Add the bin folder to system PATH: **C:\sonar-scanner-8.0.1\bin**

Step 5: Compile the Java Project

SonarQube needs .class files for Java analysis.

In Eclipse project folder, compile classes into bin.

(For Eclipse projects, bin is created automatically when project builds.)

Step 6: Run Sonar Scanner

Open Command Prompt inside the project folder:

Example: **C:\Users\...\eclipse-workspace\AddressBookSystem>**

Run:

```
sonar-scanner.bat -D"sonar.projectKey=address-book" -D"sonar.sources=src"
-D"sonar.java.binaries=bin" -D"sonar.host.url=http://localhost:9000" -
D"sonar.token=TOKEN"
```

Step 7: View the SonarQube Report

After execution, open: <http://localhost:9000/dashboard?id=address-book>

This displays:

- Code Smells
- Bugs
- Vulnerabilities
- Quality Gate Status

4) Methodology / Approach Used (Classroom Method)

Approach Followed

I followed the same approach used in the classroom:

✓ Manual SonarQube Integration using SonarScanner CLI

This method includes:

1. Running SonarQube Server locally
2. Creating project in SonarQube UI
3. Generating token
4. Running SonarScanner command from project folder
5. Viewing analysis report in SonarQube dashboard

Note about Different Approaches

Different approaches can also be used to integrate SonarQube, such as:

- SonarQube integration using **Maven**
- SonarQube integration using **Gradle**
- SonarQube integration using **Jenkins CI/CD**
- SonarLint plugin in Eclipse for live analysis

But in this project, I used the **classroom method: SonarScanner CLI**.

5) SonarQube Analysis Result Summary

From the report:

✓ Quality Gate: **PASSED**

✓ Security Issues: **0**

✓ Reliability Issues: **0**

⚠ Maintainability Issues: **62 Code Smells**

📊 Coverage: **0%** (JUnit not added)

🔄 Duplications: **8.3%**